S04 Scene Management

CRE132 worksheet

What you'll make

Two levels, a **Meadow** and a **Cave**, joined by glowing rune stones. Walk into a stone and the screen fades to black, shows **Loading...** with a progress bar, and fades in on the other level. The score carries across both.

By the end you'll know how to load scenes by name, what the **build list** is for, how a **coroutine** runs one method over many frames, and how to load a scene in the background with `LoadSceneAsync`.

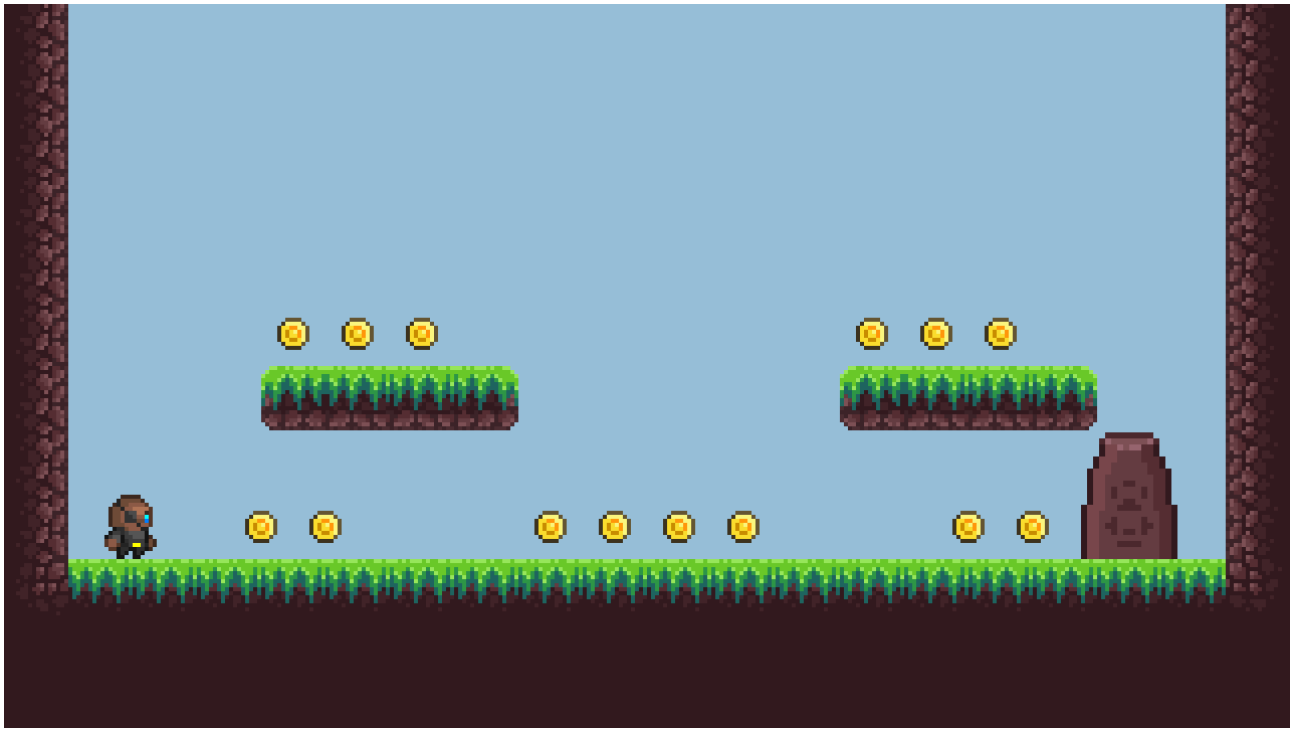


Figure 1: The finished Meadow: the player, coins on the ground and on two platforms, and the rune stone on the right.

Before you start

1. Unzip `CRE132_S04_SceneManagement_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S04_SceneManagement_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S04_SceneManagement` (the Meadow). The Cave is `S04_SceneManagement_Cave` in the same folder.
3. Press **Play**. You can walk, run, jump and collect coins, and the score shows in the top right. The rune stone (**Portal to Cave**) glows but does nothing yet. Press Play again to stop.

Both scenes have a **GameManager** and a **Scene Loader**, made from the prefabs in `Assets/_game/Prefabs/Managers`. The Scene Loader has its fade screen ready, but no script yet.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The build list

Open **File > Build Profiles** and look at the **Scene List**. Both `S04_SceneManagement` and `S04_SceneManagement_Cave` are there and ticked. Loading a scene by name only finds scenes in this list. For your own scenes, open each one and click **Add Open Scenes**, or drag them in from the Project window.

You could load the Cave with one line, `SceneManager.LoadScene("S04_SceneManagement_Cave")`, as S02 did when reloading. That works, but it cuts instantly. The Scene Loader below adds a fade and a progress bar.

2. The Scene Loader

Look inside the **Scene Loader** prefab first: double-click `Scene Loader` in `Assets/_game/Prefabs/Managers`. It has a **Fade Canvas** with **Sort Order** 100, so it draws on top of every other canvas. Inside is **Fade**, a black panel with a **Canvas Group** (Alpha 0, so you can't see it), and on the panel a **Progress Bar** (with a **Fill**) and a **Loading Text**.

Create a script `SceneLoader` in `Assets/_game/Scripts/Managers`:

```
// Loads scenes with a fade to black and a progress bar. Like the GameManager, there
// is one of it and it survives scene changes, so the fade can carry on over the new
// scene.
using System.Collections;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class SceneLoader : MonoBehaviour
{
    public static SceneLoader Instance { get; private set; }

    // The black screen and everything on it. A Canvas Group fades all its children
    // together.
    [SerializeField] private CanvasGroup fadeGroup;
    [SerializeField] private Image progressFill;
    [SerializeField] private float fadeTime = 0.5f;

    // Stops a second load starting while one is still going (the player can touch a
    // door twice).
    private bool isLoading;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);

        fadeGroup.alpha = 0f;
        fadeGroup.blocksRaycasts = false;
    }

    public void LoadScene(string sceneName)
    {
        if (isLoading)
```

```
        {
            return;
        }
        StartCoroutine(LoadSceneRoutine(sceneName));
    }

    // A coroutine can pause with yield and carry on later, so one method can run over
    // many frames.
    private IEnumerator LoadSceneRoutine(string sceneName)
    {
        isLoading = true;
        fadeGroup.blocksRaycasts = true;
        progressFill.fillAmount = 0f;

        yield return Fade(0f, 1f);

        // LoadSceneAsync loads in the background while the game keeps running, so we
        // can draw the progress bar.
        AsyncOperation loading = SceneManager.LoadSceneAsync(sceneName);
        while (!loading.isDone)
        {
            // progress goes from 0 to 0.9 while loading; the last step is switching
            // to the new scene.
            progressFill.fillAmount = loading.progress / 0.9f;
            yield return null;
        }
        progressFill.fillAmount = 1f;

        yield return Fade(1f, 0f);
        fadeGroup.blocksRaycasts = false;
        isLoading = false;
    }

    private IEnumerator Fade(float startAlpha, float endAlpha)
    {
        float time = 0f;
        while (time < fadeTime)
        {
            // Unscaled time, so fading still works if the game is paused (timeScale 0).
            time = time + Time.unscaledDeltaTime;
            fadeGroup.alpha = Mathf.Lerp(startAlpha, endAlpha, time / fadeTime);
            yield return null;
        }
        fadeGroup.alpha = endAlpha;
    }
}
```

What's going on:

- **A singleton again**, exactly like S02's GameManager: `Instance`, the duplicate check and `DontDestroyOnLoad`. It has to survive the load, or it would be destroyed halfway through its own fade.
- **Coroutines** are methods that return `IEnumerator` and are started with `StartCoroutine`. `yield return null` waits one frame and carries on. `yield return Fade(...)` waits until the whole fade has finished.
- **The fade** changes the Canvas Group's `alpha` from 0 (invisible) to 1 (black) over `fadeTime` seconds, using `Mathf.Lerp`. `blocksRaycasts` stops clicks reaching the UI underneath while the screen is dark.

- **`LoadSceneAsync`** loads the scene in the background and gives back an `AsyncOperation`. The loop sets the bar from `progress` every frame until `isDone`. Progress stops at 0.9 until the new scene takes over, so dividing by 0.9 makes the bar reach the end.
- **`isLoading`** ignores a second request while a load is going. Without it, touching a door for two frames would start two loads.

Add it: with the **Scene Loader** prefab still open, select its root object **Scene Loader** and add **Scene Loader**. Drag **Fade** into **Fade Group** and **Fill** (inside Progress Bar) into **Progress Fill**. Leave **Fade Time** at 0.5. Save the prefab and leave prefab mode with the **<** arrow at the top of the Hierarchy. Because you changed the prefab, both scenes get the script.

3. The door: SceneDoor

Make a folder `Assets/_game/Scripts/S04_SceneManagement` and create `SceneDoor` in it:

```
// A door (trigger) that takes the player to another scene by name.
// The scene must be in the build list: File > Build Profiles > Scene List.
using UnityEngine;

public class SceneDoor : MonoBehaviour
{
    [SerializeField] private string sceneToLoad;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            SceneLoader.Instance.LoadScene(sceneToLoad);
        }
    }
}
```

- **`sceneToLoad`** is a string you type in the Inspector, so one script works for every door.
- **The door doesn't load anything itself.** It asks the one SceneLoader to do it, so every door gets the same fade.

Add it: in the Meadow scene, select **Portal to Cave**, add **Scene Door** and type `S04_SceneManagement_Cave` into **Scene To Load**. Save the scene. Open `S04_SceneManagement_Cave`, select **Portal to Meadow**, add **Scene Door** with `S04_SceneManagement`, and save. Scene names must match exactly: copy them from the Project window (select the scene, press F2, Ctrl+C).

Test it: open the Meadow again and press Play. Collect a few coins and walk into the stone. The screen fades, the Cave fades in, and the score is still there. Walk into the Cave's stone to go back: the coins in the Meadow have come back. During Play, look under **DontDestroyOnLoad** in the Hierarchy: one GameManager and one Scene Loader, however many times you switch.

Check your work

- ☐ Walking into a stone fades to black and fades in on the other scene.
- ☐ The score carries across both scenes; the coins come back each time.
- ☐ There's only ever one GameManager and one Scene Loader under **DontDestroyOnLoad**.
- ☐ Compare with `CRE132_S04_SceneManagement_Finished` if something behaves differently.

Key points

- **Anything that should last across scenes needs `DontDestroyOnLoad`**, and a duplicate check, because each scene brings its own copy.

- **Coroutines** run over many frames: `yield return null` waits a frame, `yield return` another coroutine waits for it to finish.
- **Only the GameManager's data carries over.** Coins and the player belong to the scene and reset when it loads.
- **These scenes are tiny**, so the bar fills almost at once. It matters in real games with big scenes.

If something goes wrong

- **"Scene … couldn't be loaded because it has not been added to the build settings":** the name in **Scene To Load** has a typo, or the scene isn't in **File > Build Profiles > Scene List**.
- **NullReferenceException in SceneDoor:** `SceneLoader.Instance` is `null`. The Scene Loader prefab has no SceneLoader component, or you added it to the scene copy instead of the prefab and are now in the other scene.
- **NullReferenceException in SceneLoader's Awake:** the **Fade Group** or **Progress Fill** slot is empty.
- **Walking into the stone does nothing:** the Player isn't tagged **Player**, or you forgot to save the scene after adding Scene Door.

Try it

1. Make the fade slower: **Fade Time** 1.5 on the Scene Loader prefab.
2. Make the fade white: change the colour of the **Fade** image in the prefab.
3. See the progress bar: add `yield return new WaitForSeconds(1f);` just after the `while` loop (only for the demo, then take it out).
4. Harder: make the player arrive next to the stone in the scene they came back to. (Hint: store the door's name in the GameManager before loading, and move the player in `Start` in the new scene.)